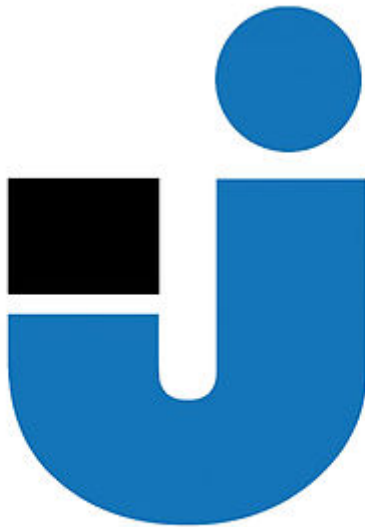


Informe de Trabajo Final

Conquista Planetaria



Universidad Nacional
ARTURO JAURETCHE

- Alumno/a: Nicolosi Giuliana, 74163.
- Profesor: Leonardo Javier Amet
- Materia: Complejidad Temporal, Estructuras de Datos y Algoritmos
- Carrera: Ingeniería en Informática
- Ciclo lectivo: 2° cuatrimestre del 2023
- Fecha: 20/11/2023

Índice

1. Introducción.....	1
2. Desarrollo	
a. Diagrama de clases UML.....	1
b. Detalles de implementación	
i. Imágenes del código.....	3
ii. Problemas encontrados con su solución.....	15
iii. Mejoras para posible actualización.....	15
3. Conclusión.....	16

Introducción

En el presente informe se presenta el trabajo final realizado para una empresa informática.

El objetivo de este trabajo fue implementar una estrategia de ataque para el Bot del juego de Conquista Planetaria cuando en el mapa se disponen los planetas jerárquicamente o en forma de árbol general.

En líneas generales, se trata de un enfrentamiento entre dos jugadores que se distinguen con dos colores: rojo para el usuario y azul para el Bot. El blanco se asigna a planetas no conquistados. Todos contienen una cantidad de naves. A diferencia del bot y el usuario, los neutrales no generan tropas.

Ambos competidores persiguen a su rival con la finalidad de “eliminarlo”. Esto sucede cuando contiene las tropas mayores al afectado y existe una arista que los conecte. En caso de no existir la conexión, se irá conquistando planetas adyacentes para así llegar a establecerse. En consecuencia, se posee la capacidad de descartarlo. Sin embargo, se debe tener en cuenta que para ganar, debe eliminarse al enemigo por completo.

Desarrollo

El siguiente diagrama de clases UML describe la estructura del sistema.



 Universidad Nacional ARTURO JAURETCHÉ	Complejidad Temporal, Estructuras de Datos y Algoritmos Informe de Trabajo Final 2do cuatrimestre del 2023	Nicolosi Giuliana. Legajo: 74163. Comisión: 3.
---	---	---

Para observar de forma más clara la clase en la que hubo cambios:

Estrategia
<pre> +Consulta1(arbol:ArbolGeneral<Planeta>): string +Consulta2(arbol:ArbolGeneral<Planeta>): string +Consulta3(arbol:ArbolGeneral<Planeta>): string +CalcularMovimiento(arbol:ArbolGeneral<Planeta>): Movimiento -nivelesPoblacion (arbol:ArbolGeneral<Planeta>): List<Planeta> -nivelBot(arbol:ArbolGeneral<Planeta>): int -hijosBot(arbol:ArbolGeneral<Planeta>): string -preOrdenString(arbol:ArbolGeneral<Planeta>): string -BuscarCaminoHaciaBot(arbol:ArbolGeneral<Planeta>): List<Planeta> -BuscarCaminoHaciaBotAux(arbol:ArbolGeneral<Planeta>,CaminoHaciaBot: List<Planeta>) -BuscarCaminoHaciaJugador(arbol:ArbolGeneral<Planeta>): List<Planeta> -BuscarCaminoHaciaJugadorAux(arbol:ArbolGeneral<Planeta>,CaminoHaciaBot: List<Planeta>): bool -BuscarCaminoBotJugador(CaminoHaciaBot:List<Planeta>,CaminoHaciaJugador:List<Planeta>) : List<Planeta> -BuscarCaminoHaciaBotEstrategia(arbol:Planeta): List<Planeta> -PlanetasBot(arbol:ArbolGeneral<Planeta>, planetasBot:List<Planeta>): void -PlanetasPoblacionMax(planetas:List<Planeta>): Planeta -BuscarCaminoHaciaBotEstrategiaAux(arbol:ArbolGeneral<Planeta>, CaminoHaciaBot:List<Planeta>, Maximo:Planeta): bool -CalcularEstrategia(CaminoHaciaBot:List<Planeta>,CaminoHaciaJugador:ArbolGeneral<Planeta>): List<Planeta> </pre>

Detalles de la implementación

i. Imágenes del código

La clase Estrategia se encarga de calcular el movimiento apropiado para el Bot en un juego de conquista planetaria. Se ha propuesto implementar los métodos de la misma, estos fueron:

1. **CalcularMovimiento** (ArbolGeneral arbol):

Este método calcula y retorna el movimiento apropiado según el estado del juego. El estado del juego es recibido en el parámetro arbol de tipo ArbolGeneral. Cada nodo del árbol contiene como dato un planeta del juego.

2. **Consulta1** (ArbolGeneral arbol):

Calcula y retorna un texto con la distancia del camino que existe entre el planeta del Bot y la raíz.

3. **Consulta2** (ArbolGeneral arbol):

Retorna un texto con el listado de los planetas ubicados en todos los descendientes del nodo que contiene al planeta del Bot.

4. **Consulta3** (ArbolGeneral arbol):

Calcula y retorna en un texto la población total y promedio por cada nivel del árbol.

```
public String Consulta1(ArbolGeneral<Planeta> arbol)
{
    string msj = "La distancia entre el planeta del Bot y la raíz es " + nivelBot(arbol);

    return msj;
}

private int nivelBot(ArbolGeneral<Planeta> arbol)
{
    // Devuelve la profundidad a la que se encuentra el planeta del Bot en el árbol
    // Creación de una cola para realizar un recorrido por niveles con separadores
    Cola<ArbolGeneral<Planeta>> c = new Cola<ArbolGeneral<Planeta>>();
    ArbolGeneral<Planeta> arbolAux;

    // Se encola el árbol raíz y un separador (null) para indicar el nivel
    c.encolar(arbol);
    c.encolar(null);

    //Variable de nivel
    int nivel = 0;

    //Mientras cola no este vacia
    while (!c.esVacia())
    {
        //Desencolo elemento en arbol aux
        arbolAux = c.desencolar();

        // Si es un separador
        if (arbolAux == null)
        {
            if (!c.esVacia())
            {
                // Encolamos otro separador para indicar el siguiente nivel
                c.encolar(null);
                // Incrementamos el nivel porque se avanza al siguiente nivel
                nivel++;
            }
        }
        // Si es un nodo (arbol)
        else
        {
            // Se procesa el nodo actual

            // Si se encuentra el planeta del Bot
            if (arbolAux.getDatoRaiz().EsPlanetaDeLaIA())
                //Devolvemos el nivel actual
                return nivel;

            // Si no, se encola los hijos del nodo para explorar niveles inferiores
            foreach (var hijo in arbolAux.getHijos())
                c.encolar(hijo);
        }
    }
    // Si no se encontró el planeta del Bot en el árbol, se devuelve el nivel como -1
    return -1;
}
```

Consulta1 devuelve un string que contiene la distancia entre el planeta del Bot y la raíz.

 Universidad Nacional ARTURO JAURETCHÉ	Complejidad Temporal, Estructuras de Datos y Algoritmos Informe de Trabajo Final 2do cuatrimestre del 2023	Nicolosi Giuliana. Legajo: 74163. Comisión: 3.
---	--	---

Para esto, se invoca a **nivelBot**, un método auxiliar. El mismo realiza un recorrido por niveles con separadores.

Existen dos opciones, que el elemento sea un separador (null) o un nodo.

Si es un separador, se incrementa la variable nivel.

Si es un nodo, se obtiene su origen y se verifica que sea del BOT. Si es, se retorna su nivel y caso contrario, se encolan sus hijos para procesarlos y así, recorrer los niveles inferiores.

En caso de no pertenecer a ninguna condición, se devuelve un valor x. En la realidad, esto no sucede puesto a que siempre va a existir un planeta del bot, solo sucederá cuando el usuario gane. En ese momento, el juego termina. Por lo tanto, nunca se retornará realmente ese valor si no que es una obligación puesto a que la función devuelve un int.

```

public String Consulta2(ArbolGeneral<Planeta> arbol)
{
    String msj = "\nPlanetas descendientes del Bot: " + hijosBot(arbol)

    return msj;
}

//===== METODOS PARA CONSULTA 2 =====
private string hijosBot(ArbolGeneral<Planeta> arbol)
{
    // Recorrido por niveles utilizando una cola
    Cola<ArbolGeneral<Planeta>> c = new Cola<ArbolGeneral<Planeta>>();
    ArbolGeneral<Planeta> arbolAux;
    string bothijos = "";

    // Encolamos el árbol raíz
    c.encolar(arbol);

    // Realizamos el recorrido por niveles
    while (!c.esVacia())
    {
        //Desencolo elemento en arbol aux
        arbolAux = c.desencolar();

        // Si se encuentra el primer planeta que es del bot
        if (arbolAux.getDatoRaiz().EsPlanetaDeLaIA())
        {
            // Realizamos un recorrido preorden para obtener la población de los planetas
            bothijos = preOrdenString(arbolAux);

            // Se busca y saca el primer elemento de bothijos (La RAIZ, o sea el planeta bot)
            int primerEspacio = bothijos.IndexOf(" ");
            bothijos = bothijos.Remove(0, primerEspacio + 1);

            // Se retorna la representación de la población de los planetas en el subárbol
            return bothijos;
        }
    }
}

```

 Universidad Nacional ARTURO JAURETCHÉ	Complejidad Temporal, Estructuras de Datos y Algoritmos Informe de Trabajo Final 2do cuatrimestre del 2023	Nicolosi Giuliana. Legajo: 74163. Comisión: 3.
--	---	--

```

        // Si no, se encola los hijos del árbol actual para seguir la búsqueda
        foreach (var hijo in arbolAux.getHijos())
        {
            c.encolar(hijo);
        }
    }
    // Si no se encuentra ningún planeta BOT, se retorna una cadena vacía
    return botHijos;
}

private string preOrdenString(ArbolGeneral<Planeta> arbol)
{
    //Declara la variable recorrido que se utiliza para almacenar la población del árbol en cadena
    StringBuilder recorrido = new StringBuilder();

    // Se procesa la población del planeta raíz primero; se agrega población de la raíz al recorrido
    recorrido.Append(arbol.getDatoRaiz().Poblacion().ToString() + " ");

    // Despues se procesa los hijos recursivamente en preorden
    //Para cada hijo en el arbol; lo agrego al recorrido
    foreach (var hijo in arbol.getHijos())
    {
        recorrido.Append(preOrdenString(hijo));
    }

    // Retorno la representación de la población en forma de cadena
    return recorrido.ToString();
}

```

Consulta2 devuelve un string donde se informan los hijos del BOT.

Se emplea el método **hijosBOT**. Este último realiza un recorrido por niveles parecido al de la anterior Consulta1.

Si es el primer nodo del BOT, se obtiene su origen. Se propone la realización de realizar un recorrido preorden (por elección propia) para recorrer en primer lugar, la raíz y posteriormente sus hijos. Tal recorrido retorna la representación de la población en forma de cadena. Luego, se elimina a la raíz (y el espacio siguiente) porque solo se quieren listar los hijos descendientes.

Si no se encuentra al primer nodo, se procesan los hijos del árbol actual para continuar con la búsqueda deseada.

Al igual que en la Consulta1 si no se cumple ninguna especificación, de todas formas debe retornar algo. Entonces, devuelve una cadena vacía.

```
public String Consulta3(ArbolGeneral<Planeta> arbol)
{
    //Variable que almacena cadena
    String msj = "";
    //Variable que almacena el número del nivel actual.
    int i = 0;

    //Se recorre una lista de arrays donde cada array representa un nivel.
    //1er elemento de cada array corresponde a la población total del nivel.
    //2do elemento de cada array corresponde al promedio de la población por planeta de ese nivel.
    //Recorro lista de arreglos
    foreach (int[] nivel in nivelesPoblacion(arbol))
    {
        msj += "\n\n Nivel " + i + ": " + "la población total es " + nivel[0] + " y la población promedio por planeta es " + nivel[1];
        i++;
    }
    return msj;
}

//===== METODO PARA CONSULTA 3 =====
private List<int[]> nivelesPoblacion(ArbolGeneral<Planeta> arbol)
{
    // Declaración de una cola para el recorrido por niveles
    Cola<ArbolGeneral<Planeta>> c = new Cola<ArbolGeneral<Planeta>>();
    // Lista para almacenar la información de población por nivel
    List<int[]> nivelesPobla = new List<int[]>();
    ArbolGeneral<Planeta> arbolAux;

    // Encolamos el árbol raíz y un separador (null) para indicar el nivel
    c.encolar(arbol);
    c.encolar(null);

    int sumaPoblacion = 0;
    int cantPlanetas = 0;

    // Comienza el recorrido por niveles
    while (!c.esVacia())
    {
        arbolAux = c.desencolar();
        // Si es un separador
        if (arbolAux == null)
        {
            if (!c.esVacia())
            {
                // Encolamos otro separador para indicar el siguiente nivel
                c.encolar(null);

                // Calculamos la población total y el promedio de población por planeta del nivel actual
                int[] nivelPobla = { sumaPoblacion, sumaPoblacion / cantPlanetas };
                nivelesPobla.Add(nivelPobla);

                // Reiniciamos las variables para el próximo nivel
                sumaPoblacion = 0;
                cantPlanetas = 0;
            }
        }
        // Si es un nodo (arbol)
        else
        {
            // Procesamos la población del planeta actual
            sumaPoblacion += arbolAux.getDatoRaiz().Poblacion();
            cantPlanetas++;

            // Encolamos los hijos del planeta para explorar niveles inferiores
            foreach (var hijo in arbolAux.getHijos())
            {
                c.encolar(hijo);
            }
        }
    }

    // Al finalizar el recorrido, agregamos la información del último nivel (raíz)
    int[] nivelPoblaFinal = { sumaPoblacion, sumaPoblacion / cantPlanetas };
    nivelesPobla.Add(nivelPoblaFinal);

    // Se retorna la lista de población por nivel
    return nivelesPobla;
}
```


 Universidad Nacional ARTURO JAURETCHÉ	Complejidad Temporal, Estructuras de Datos y Algoritmos Informe de Trabajo Final 2do cuatrimestre del 2023	Nicolosi Giuliana. Legajo: 74163. Comisión: 3.
---	--	---

Consulta3 retorna como string la población total y el promedio de población en cada nivel del árbol. Para esto, recorre una lista de arreglos que contiene un arreglo por nivel. Este arreglo contiene sus dos informaciones específicas.

Se invoca el método de **nivelesPoblacion**, el cual propone un recorrido por niveles con separadores y así:

Si es un separador y la cola no está vacía, entonces se procede a calcular la variable nivelPobla que contiene la población por nivel y agregarlo a la lista que va a ser el resultado.

Si es un nodo, se calcula la suma de la población (**sumaPoblacion**) y la cantidad de planetas (**cantPlanetas**). Luego, se recorren los siguientes hijos.

Una vez terminado el recorrido, se agrega la información del último nivel a la lista y se la retorna completa.

```

public Movimiento CalcularMovimiento(ArbolGeneral<Planeta> arbol)
{
    // Se calculan los caminos hacia el bot y el jugador y se los combina
    List<Planeta> caminoHaciaBot = BuscarCaminoHaciaBot(arbol);
    List<Planeta> caminoHaciaJugador = BuscarCaminoHaciaJugador(arbol);
    List<Planeta> caminoBotJugador = BuscarCaminoBotJugador(caminoHaciaBot, caminoHaciaJugador);

    // Estrategia cuando el Bot se encuentra al lado del jugador
    if (caminoBotJugador[1].EsPlanetaDelJugador() && caminoBotJugador[0].Poblacion() / 2 < caminoBotJugador[1].Poblacion())
    {
        // Si el planeta adyacente al bot pertenece al jugador y la población del bot es menor que la mitad de la población del jugador,
        //se implementa una estrategia de refuerzo.
        caminoHaciaBot = BuscarCaminoHaciaBotEstrategia(arbol);
        //Camino desde el bot con mas naves
        caminoBotJugador = CalcularEstrategia(caminoHaciaBot, caminoHaciaJugador);
    }
    //Se realiza un movimiento entre el bot y el planeta adyacente al mismo
    Movimiento ataque = new Movimiento(caminoBotJugador[0], caminoBotJugador[1]);

    return ataque;
}

```

CalcularMovimiento emplea 5 métodos. Su función es retornar el movimiento correspondiente dependiendo del estado del juego.

En primer lugar, se invocan dos (**BuscarCaminoHaciaBot**, **BuscarCaminoHaciaJugador**) para generar dos listas de planetas: uno que posee los planetas que se encuentran desde la raíz hacia el BOT y otro desde la raíz al jugador.

Posteriormente, se llama al tercer método **BuscarCaminoBotJugador** el cual toma como parámetros las dos listas anteriores y las combina en un solo camino para representar la ruta desde el Bot hasta el jugador.

 Universidad Nacional ARTURO JAURETCHÉ	Complejidad Temporal, Estructuras de Datos y Algoritmos Informe de Trabajo Final 2do cuatrimestre del 2023	Nicolosi Giuliana. Legajo: 74163. Comisión: 3.
--	---	--

Se procede a analizar el estado del juego para tomar acción en caso de que se necesiten aplicar refuerzos. La estrategia del Bot actúa cuando se encuentra al lado de un planeta del jugador y la mitad de la población del bot, es la cantidad de naves que debe enviar, es menor que la población que hay en el planeta del jugador. Esta implica el uso de otros dos métodos auxiliares. Entonces se emplea **BuscarCaminoHaciaBotEstrategia** y **CalcularEstrategia** que devuelven una lista de los planetas azules con más tropas desde el origen hasta la raíz y una lista que contiene a los planetas del Bot con más tropas hasta el jugador rojo.

Por último, se procede a concluir un ataque desde planeta del bot con posición 0 hacia otro con posición 1 que puede ser tanto para otro planeta del Bot (refuerzo a sí mismo), un planeta neutral (conquista) o para intentar derribar al enemigo (conquista) mediante la lista **CaminoBotJugador**, la cual contiene la estrategia.

Seguidamente, se muestra la implementación de los métodos en uso.

```
//===== METODOS PARA CALCULAR MOVIMIENTO =====
//Se arma una lista con el camino desde la raíz hasta el primer planeta Bot
private List<Planeta> BuscarCaminoHaciaBot(ArbolGeneral<Planeta> arbol)
{
    // Se crea una lista para guardar el camino hacia el planeta del Bot
    List<Planeta> CaminoHaciaBot = new List<Planeta>();

    // Llamo a la función auxiliar para hacer la búsqueda
    BuscarCaminoHaciaBotAux(arbol, CaminoHaciaBot);

    // Retorno la lista
    return CaminoHaciaBot;
}

private bool BuscarCaminoHaciaBotAux(ArbolGeneral<Planeta> arbol, List<Planeta> CaminoHaciaBot)
{
    if (arbol == null)
        return false;

    // Se agrega el planeta actual al camino
    CaminoHaciaBot.Add(arbol.getDatoRaiz());

    if (arbol.getDatoRaiz().EsPlanetaDeLaIA())
    {
        // Se encontró el planeta del Bot, se termina la búsqueda
        return true;
    }

    // Si no, se recorre los hijos del planeta actual
    foreach (var hijo in arbol.getHijos())
    {
        //Si se encontró el planeta del Bot en el subárbol, retornamos true
        if (BuscarCaminoHaciaBotAux(hijo, CaminoHaciaBot))
        {
            return true;
        }
    }

    // Si no se encontró el planeta en el subárbol, se eliminan los planetas agregados en la lista
    CaminoHaciaBot.RemoveAt(CaminoHaciaBot.Count - 1);
}

//Si se retorna false
return false;
```

 Universidad Nacional ARTURO JAURETCHÉ	Complejidad Temporal, Estructuras de Datos y Algoritmos Informe de Trabajo Final 2do cuatrimestre del 2023	Nicolosi Giuliana. Legajo: 74163. Comisión: 3.
--	---	--

Los métodos auxiliares **BuscarCaminoHaciaBot**, **BuscarCaminoHaciaBotAux**, **BuscarCaminoHaciaJugador** y **BuscarCaminoHaciaJugadorAux**, se encargan de encontrar un camino desde el planeta raíz del árbol hasta un destino pudiendo pertenecer al jugador o al Bot. Los dos primeros actúan de la misma forma que los restantes con la diferencia de que unos buscan hacia el Bot y otros hacia el jugador.

BuscarCaminoHaciaBot devuelve la lista **CaminoHaciaBot** que contiene el camino desde la raíz hasta el primer planeta Bot empleando **BuscarCaminoHaciaBotAux**.

Este último método toma distintas operaciones según el nodo a procesar.

Se lo agrega a la lista **CaminoHaciaBot**. En primera instancia se verifica si es del bot; si lo es, se detiene la búsqueda y se obtiene en booleano verdadero para indicar que se ha encontrado un planeta. Si no lo es, se procesan sus hijos de forma recursiva, es decir, con la invocación del mismo método.

Como consecuencia, si se encuentra el planeta del Bot en un hijo, se retorna true. En caso contrario, se suprime de la lista **CaminoHaciaBot** al último planeta agregado para continuar la búsqueda en otro subárbol.

A la larga, si no se encuentra el planeta del Bot se retorna false. Esto no sucede a menos que el jugador gane.

```
//Buscar camino hacia el jugador (hace lo mismo que el camino hacia el bot,
//una lista con todos los planetas desde la raíz hasta el primer jugador)
private List<Planeta> BuscarCaminoHaciaJugador(ArbolGeneral<Planeta> arbol)
{
    List<Planeta> CaminoHaciaJugador = new List<Planeta>();

    BuscarCaminoHaciaJugadorAux(arbol, CaminoHaciaJugador);

    return CaminoHaciaJugador;
}

private bool BuscarCaminoHaciaJugadorAux(ArbolGeneral<Planeta> arbol, List<Planeta> CaminoHaciaJugador)
{
    CaminoHaciaJugador.Add(arbol.getDatoRaiz());

    if (arbol.getDatoRaiz().EsPlanetaDelJugador()) // Se encontró el planeta del Bot
        return true;

    foreach (var hijo in arbol.getHijos())
    {
        if (BuscarCaminoHaciaJugadorAux(hijo, CaminoHaciaJugador))
            return true;

        CaminoHaciaJugador.RemoveAt(CaminoHaciaJugador.Count - 1);
    }
    return false;
}
```

 Universidad Nacional ARTURO JAURETCHÉ	Complejidad Temporal, Estructuras de Datos y Algoritmos Informe de Trabajo Final 2do cuatrimestre del 2023	Nicolosi Giuliana. Legajo: 74163. Comisión: 3.
---	--	---

BuscarCaminoHaciaJugador y **BuscarCaminoHaciaJugadorAux** se emplea de la misma forma que los dos anteriores solo que en vez de buscar el camino hacia el Bot, lo hace hacia el jugador.

```
// Función para combinar los caminos del bot y el jugador.
private List<Planeta> BuscarCaminoBotJugador(List<Planeta> caminoHaciaBot, List<Planeta> caminoHaciaJugador)
{
    List<Planeta> caminoBotJugador = new List<Planeta>();
    // Se recorre el camino hacia el bot de forma inversa y se agrega al caminoBotJugador.
    for (int i = caminoHaciaBot.Count - 1; i >= 0; i--)
    {
        caminoBotJugador.Add(caminoHaciaBot[i]);
    }
    // Se recorre el camino hacia el jugador para agregarlo al caminoBotJugador.
    foreach (var planeta in caminoHaciaJugador)
    {
        // Si el planeta es del Bot, lo coloca en la primera posición de caminoBotJugador.
        if (planeta.EsPlanetaDeLaIA())
        {
            //El camino va desde el bot mas cercano al jugador
            caminoBotJugador[0] = planeta;
        }
        // Si el planeta es neutral o del jugador,
        else
        {
            //Lo agrega a la siguiente posición de caminoBotJugador.
            caminoBotJugador.Add(planeta);
        }
    }
    // Retorna el camino combinado.
    return caminoBotJugador;
}
```

El método **BuscarCaminoBotJugador** une los caminos del BOT y el jugador, creando una nueva lista llamada **caminoBotJugador** que contiene los planetas que se encuentran entre medio de ellos.

Primero, se crea una lista vacía **caminoBotJugador** para almacenar el camino resultante. Luego, se recorre la lista **caminoHaciaBot** en orden inverso, agregando cada planeta a **caminoBotJugador**.

A continuación, se recorre la lista **caminoHaciaJugador**. Para cada planeta, se verifica si pertenece al BOT. Si es así, se coloca en la primera posición de **caminoBotJugador**, asegurando que el camino siempre vaya desde el último BOT encontrado, es decir, el más cercano al jugador. Si el planeta es neutral o pertenece al jugador, se agrega a la lista en la siguiente posición.

Finalmente, el método retorna **caminoBotJugador**.

Con lo que respecta a la estrategia de refuerzo, también se utilizan varios metodos.

 Universidad Nacional ARTURO JAURETCHÉ	Complejidad Temporal, Estructuras de Datos y Algoritmos Informe de Trabajo Final 2do cuatrimestre del 2023	Nicolosi Giuliana. Legajo: 74163. Comisión: 3.
--	---	--

```
// ESTRATEGIA DE REFUERZO (cuando no alcanzan las naves del bot para atacar al jugador)
private List<Planeta> BuscarCaminoHaciaBotEstrategia(ArbolGeneral<Planeta> arbol)
{
    List<Planeta> CaminoHaciaBot = new List<Planeta>();
    List<Planeta> planetasBot = new List<Planeta>();

    // Se obtienen los planetas del BOT y se guardan en la lista planetasBot
    PlanetasBot(arbol, planetasBot);

    // Encontrar el planeta bot con la mayor cantidad de naves
    Planeta planetaMax = PlanetaPoblacionMax(planetasBot);

    // Se busca el camino hacia el planeta del Bot con la mayor cantidad de naves y se guarda en la lista CaminoHaciaBot
    BuscarCaminoHaciaBotEstrategiaAux(arbol, CaminoHaciaBot, planetaMax);

    // Retorna la lista del camino calculado anteriormente
    return CaminoHaciaBot;
}
```

El método **BuscarCaminoHaciaBotEstrategia** crea una lista que representa el camino desde el planeta raíz hasta el planeta del BOT con la mayor cantidad de tropas.

Primero, se inicializan dos listas vacías, **caminoHaciaBot** y **planetasBot**.

Luego, se invoca el método **PlanetasBot** para obtener una lista con los planetas que pertenecen al BOT.

A continuación, se invoca el método **PlanetaPoblacionMax** para identificar el planeta del BOT con la mayor cantidad de naves.

Finalmente, se invoca el método **BuscarCaminoHaciaBotAux** para ajustar la lista **caminoHaciaBot** agregando los planetas que se encuentran en el camino desde la raíz hasta el planeta del BOT con la mayor cantidad de naves.

```
//Funcion recursiva para encontrar los planetas del bot
private void PlanetasBot(ArbolGeneral<Planeta> arbol, List<Planeta> planetasBot)
{
    // Si el planeta actual es del bot
    if (arbol.getDatoRaiz().EsPlanetaDeLaIA())
    {
        // Lo agrega a la lista de planetas
        planetasBot.Add(arbol.getDatoRaiz());
    }
    foreach (var hijo in arbol.getHijos())
        // Llama recursivamente a la funcion para los hijos del árbol
        PlanetasBot(hijo, planetasBot);
}
```

El método **PlanetasBot** busca y almacena en una lista todos los planetas que pertenecen al BOT mediante una adaptación del recorrido preOrden.

El método funciona de forma recursiva, evaluando cada planeta en la estructura del árbol.

Si un planeta es propiedad del BOT, se agrega a la lista **planetasBot**.

Luego, el método se llama recursivamente para procesar a los hijos del planeta actual.

 Universidad Nacional ARTURO JAURETCHÉ	Complejidad Temporal, Estructuras de Datos y Algoritmos Informe de Trabajo Final 2do cuatrimestre del 2023	Nicolosi Giuliana. Legajo: 74163. Comisión: 3.
--	---	--

De esta manera, el método busca todos los planetas que pertenecen al BOT en todos los niveles del árbol.

```
private Planeta PlanetaPoblacionMax(List<Planeta> planetas)
{
    int max = -1;
    //Recorro la lista de planetas
    foreach (var planeta in planetas)
    {
        // Si la poblacion del planeta actual es mayor a max
        if (planeta.Poblacion() > max)
            //El max va a pasar a ser el numero de naves de ese planeta
            max = planeta.Poblacion();
    }
    //Luego se busca al planeta que tenga la poblacion igual a max
    return planetas.Find(p => p.Poblacion() == max);
}
```

El método **PlanetaPoblacionMax** recibe una lista de planetas y retorna el planeta con la mayor cantidad de naves.

Se inicializa una variable max con un valor de -1. Esta variable almacenará el valor máximo de naves encontrado.

Se recorre la lista de planetas. En cada iteración, se compara la cantidad de naves del planeta actual con el valor almacenado en max. Si la cantidad de naves es mayor, se actualiza el valor de max.

Se busca el planeta en la lista que tenga una cantidad de naves igual a max. Este planeta es el que tiene la mayor cantidad de naves.

```
private bool BuscarCaminoHaciaBotEstrategiaAux(ArbolGeneral<Planeta> arbol, List<Planeta> CaminoHaciaBot, Planeta Maximo)
{
    //Agrega el planeta actual al camino hacia el Bot
    CaminoHaciaBot.Add(arbol.getDatosRaiz());
    //Si se encontró el planeta del Bot con mas naves, retorna true
    if (arbol.getDatosRaiz() == Maximo)
        return true;

    //Si no, recorre los hijos del planeta actual buscando al planeta Maximo
    foreach (var hijo in arbol.getHijos())
    {
        if (BuscarCaminoHaciaBotEstrategiaAux(hijo, CaminoHaciaBot, Maximo))
            return true;
        CaminoHaciaBot.RemoveAt(CaminoHaciaBot.Count - 1);
    }
    return false;
}
```

El método **BuscarCaminoHaciaBotEstrategiaAux** encuentra un camino desde el planeta raíz hasta el planeta del BOT con la mayor cantidad de naves a través de un recorrido preOrden.

Se agrega el planeta actual a la lista caminoHaciaBot.

 Universidad Nacional ARTURO JAURETCHÉ	Complejidad Temporal, Estructuras de Datos y Algoritmos Informe de Trabajo Final 2do cuatrimestre del 2023	Nicolosi Giuliana. Legajo: 74163. Comisión: 3.
--	---	--

Se verifica si el planeta actual es igual al planeta del BOT con la mayor cantidad de naves. Si lo es, el método retorna true.

Si el planeta actual no es igual al planeta del BOT con la mayor cantidad de naves, el método se llama recursivamente para cada hijo del planeta actual.

Si uno de los hijos contiene el planeta del BOT con la mayor cantidad de naves, el método retorna true.

Si ninguno de los hijos del planeta actual contiene el planeta del BOT con la mayor cantidad de naves, se elimina el último planeta de la lista caminoHaciaBot.

```
// Calcular la estrategia para el refuerzo
private List<Planeta> CalcularEstrategia(List<Planeta> caminoHaciaBot, List<Planeta> caminoHaciaJugador)
{
    // Lista para guardar el camino entre el bot y el jugador
    List<Planeta> caminoBotJugador = new List<Planeta>();

    int posPlanetaMax = caminoHaciaBot.Count - 1;
    //Es el ultimo planeta porque en CaminoHaciaBot, va desde la raiz hasta el planeta con mayor cantidad de naves
    if (!caminoHaciaJugador.Contains(caminoHaciaBot[posPlanetaMax]))
    {
        // El camino hacia el jugador no contiene al planeta del Bot con la población máxima
        // Recorre el camino hacia el Bot de forma inversa y lo agrega a caminoBotJugador
        for (int i = posPlanetaMax; i > 0; i--)
        {
            caminoBotJugador.Add(caminoHaciaBot[i]);
        }
        // Agrega el camino hacia el jugador
        caminoBotJugador.AddRange(caminoHaciaJugador);
    }
    else
    {
        // El planeta del Bot de población máxima está en el camino hacia el jugador

        caminoBotJugador.Add(caminoHaciaBot[posPlanetaMax]); // Agrega el planeta del Bot al camino

        // Recorre el camino hacia el jugador para agregarlo a caminoBotJugador, pero a partir del planeta del Bot de población máxima
        foreach (var planeta in caminoHaciaJugador)
        {
            if (!caminoHaciaBot.Contains(planeta)) //si el planeta actual no es del bot
                caminoBotJugador.Add(planeta);
        }
    }
    return caminoBotJugador; // Devuelve la estrategia de refuerzo
}
```

El método **CalcularEstrategia** recibe dos listas de planetas, **caminoHaciaBot** y **caminoHaciaJugador**. Su propósito es encontrar una ruta que vaya desde el planeta Bot con la población más alta hasta el planeta del Jugador.

Para ello, se crea una lista vacía, **caminoBotJugador**, que almacenará la ruta encontrada. A continuación, se determina la posición del planeta Bot con la población más alta y se almacena en la variable **PosPlanetaMax**. Este planeta es el último en la lista caminoHaciaBot porque la lista va desde el planeta raíz hasta él.

 Universidad Nacional ARTURO JAURETCHÉ	Complejidad Temporal, Estructuras de Datos y Algoritmos Informe de Trabajo Final 2do cuatrimestre del 2023	Nicolosi Giuliana. Legajo: 74163. Comisión: 3.
---	--	---

Se comprueba si el planeta Bot con la población más alta se encuentra en la lista **caminoHaciaJugador**. Si no es así, se recorre la lista **caminoHaciaBot** en orden inverso y se añade cada planeta visitado a la lista **caminoBotJugador**. A continuación, se añaden todos los planetas de la lista **caminoHaciaJugador** al final de **caminoBotJugador**.

Por otro lado, si el planeta está en la lista, se añade a la lista **caminoBotJugador**. A continuación, se recorre la lista **caminoHaciaJugador** y se añaden a la lista **caminoBotJugador** únicamente los planetas que no estén incluidos en la lista **caminoHaciaBot**.

Por último, la función devuelve la lista **caminoBotJugador**.

ii. Problemas encontrados con sus respectivas soluciones

En el método **CalcularMovimiento**, la condición para implementar la estrategia de refuerzo se verifica con la población total del planeta del Bot. Esto causaba un problema si el BOT tiene una población mayor que la población del jugador, pero no lo suficiente para vencerlo ya que se generaba un loop al solo poder enviar la mitad de las naves para atacar.

Para resolver este problema, se optó por dividir a la población del planeta del Bot por dos antes de verificar la condición. Esto garantiza que la estrategia de refuerzo se implemente más veces. Así, se evita que el Bot quede atrapado en un bucle infinito, atacando con las naves que tiene pero sin poder vencer al jugador.

ii. Ideas o sugerencias para posible actualización

1. Ampliar el alcance de la expansión del Bot

Permitir que el BOT se expanda más por los subárboles adyacentes. Así, el Bot conquista una mayor cantidad de planetas, lo que aumentaría su capacidad de producir naves y desarrollar estrategias de refuerzo de manera más eficiente.

2. Implementar niveles de dificultad distintos

Los jugadores podrían decidir con cuál desafío quieren enfrentarse. Los niveles se basarían en intensificar la velocidad y la cantidad de ataques del Bot.

3. Aumentar rapidez de las operaciones del Bot

Acortar el tiempo que tarda el BOT en realizar ataques para que sea más competitivo y el usuario, por capacidad humana, tarda más en efectuar cualquier operación.

 Universidad Nacional ARTURO JAURETCHE	Complejidad Temporal, Estructuras de Datos y Algoritmos Informe de Trabajo Final 2do cuatrimestre del 2023	Nicolosi Giuliana. Legajo: 74163. Comisión: 3.
---	--	---

Conclusión

El trabajo final presentado ha demostrado que la estrategia diseñada es efectiva para atacar a los planetas enemigos. La implementación y los métodos que requiere para efectuarse son producto de los contenidos vistos en la materia desde el primer día.

Para mi persona, es satisfactorio que haya podido poner a prueba y a su vez, obtener los resultados esperados.